Azure Service Bus – Peek-Lock Mode vs Receive-and-Delete Mode

1. Overview

When creating a `ServiceBusReceiver`, we can specify how messages should be received.

There are two main receive modes:

1. **Peek-Lock**
2. **Receive-and-Delete**

The default mode is **Peek-Lock** if no other mode is specified.

---

2. Peek-Lock Mode

In **Peek-Lock mode**, receiving a message does **not immediately delete it** from the queue.

The receiver gets the message and temporarily locks it.

The application must explicitly complete the message after successful processing.

```
var receiver = client.CreateReceiver(
    queueName,
    new ServiceBusReceiverOptions
    {
        ReceiveMode = ServiceBusReceiveMode.PeekLock
    });
```

Then:

```
var message =
    await receiver.ReceiveMessageAsync();
```

After successful processing:

```
await receiver.CompleteMessageAsync(message);
```

Flow

```
Queue
  ↓
Receive Message
  ↓
Message is locked
  ↓
Process Message
  ↓
CompleteMessageAsync()
  ↓
Message deleted
```

Important

If processing fails, the application does not have to complete the message. It can use appropriate message handling such as **abandon**, **defer**, or **dead-letter**.

---

3. Receive-and-Delete Mode

In **Receive-and-Delete mode**, the message is removed from the queue as soon as it is received.

There is no separate `CompleteMessageAsync()` operation required.

```
var receiver = client.CreateReceiver(
    queueName,
    new ServiceBusReceiverOptions
    {
        ReceiveMode = ServiceBusReceiveMode.ReceiveAndDelete
    });
```

Then:

```
var message =
    await receiver.ReceiveMessageAsync();
```

The message is automatically removed when it is received.

Flow

```
Queue
  ↓
Receive Message
  ↓
Message automatically deleted
  ↓
Process Message
```

Important Risk

If the application crashes immediately after receiving the message, the message has already been removed.

Therefore, the application cannot normally retry that message.

---

4. Peek-Lock vs Receive-and-Delete

Feature	Peek-Lock	Receive-and-Delete
Default mode	Yes	No
Message deleted immediately?	No	Yes
Message locked?	Yes	No
Explicit completion required?	Yes	No
Can retry after processing failure?	Yes	No
Safer for business processing?	Yes	Less safe
Suitable for critical messages?	Yes	Usually not

Easy Memory Trick

Peek-Lock → Receive → Lock → Process → Complete

Receive-and-Delete → Receive → Delete immediately

---

5. Why is Peek-Lock Usually Preferred?

Suppose we receive an important payment message:

```
Payment ₹50,000
```

Using Peek-Lock:

```
Receive
  ↓
Message locked
  ↓
Process Payment
  ↓
Success
  ↓
Complete
  ↓
Message deleted
```

If processing fails:

```
Receive
  ↓
Message locked
  ↓
Processing fails
  ↓
Do not complete
  ↓
Message can become available again
```

This provides a safer processing model for important business operations.

---

6. What is Peek?

Peek is different from **Peek-Lock**.

The purpose of Peek is simply:

> Look at a message without consuming it.

We can use the `PeekMessageAsync()` method.

```
var message =
    await receiver.PeekMessageAsync();
```

The message is returned for inspection, but it is not locked for processing.

---

7. Important Point About Peek

The `PeekMessageAsync()` method can be used regardless of whether the receiver was created using:

- Peek-Lock mode
- Receive-and-Delete mode

Example:

```
var receiver = client.CreateReceiver(
    queueName,
    new ServiceBusReceiverOptions
    {
        ReceiveMode = ServiceBusReceiveMode.ReceiveAndDelete
    });
```

We can still call:

```
var message =
    await receiver.PeekMessageAsync();
```

---

8. Can We Complete a Peeked Message?

No.

A message returned by `PeekMessageAsync()` is only being inspected.

It is not received for processing and does not have the normal receive lock associated with it.

Therefore, trying to complete a peeked message will result in an exception.

Why?

Because the intention of Peek is:

```
Just Look
   ↓
Do Not Process
   ↓
Do Not Delete
```

Whereas Complete means:

```
I Processed This Message
   ↓
Delete It
```

These two operations have different purposes.

---

9. Peek vs Receive

Feature	Peek	Receive
Purpose	Inspect message	Process message
Removes message?	No	Depends on receive mode
Locks message?	No processing lock	Peek-Lock can lock it
Can complete?	No	Yes in Peek-Lock mode
Typical use	Monitoring / inspection	Application processing

---

10. Real-Time Example

Suppose the queue contains:

```
Order #1001
Order #1002
Order #1003
```

Using Peek

We can inspect the messages:

```
Peek
 ↓
Look at Order #1001
 ↓
Message remains in queue
```

This is useful when we want to check what messages are available without processing them.

Using Peek-Lock

```
Receive
 ↓
Order #1001 locked
 ↓
Process Order
 ↓
Complete
 ↓
Order #1001 removed
```

Using Receive-and-Delete

```
Receive
 ↓
Order #1001 immediately deleted
 ↓
Process Order
```

If processing fails after deletion, the original message is no longer available for normal retry.

---

11. Practical Decision

Use Peek-Lock when:

- Message processing is important.
- Processing may fail.
- You need retry capability.
- You want to explicitly complete successful messages.
- You need abandon/defer/dead-letter handling.

Use Receive-and-Delete when:

- Losing a message after receiving it is acceptable.
- Processing is simple or non-critical.
- You intentionally want automatic deletion.

Use Peek when:

- You only want to inspect messages.
- You don't want to consume or lock them.
- You want to examine messages for monitoring or troubleshooting.

---

12. Complete Example – Peek-Lock

```
var receiver = client.CreateReceiver(
    queueName,
    new ServiceBusReceiverOptions
    {
        ReceiveMode = ServiceBusReceiveMode.PeekLock
    });

var message =
    await receiver.ReceiveMessageAsync();

// Process message

await receiver.CompleteMessageAsync(message);
```

---

13. Complete Example – Receive-and-Delete

```
var receiver = client.CreateReceiver(
    queueName,
    new ServiceBusReceiverOptions
    {
        ReceiveMode = ServiceBusReceiveMode.ReceiveAndDelete
    });

var message =
    await receiver.ReceiveMessageAsync();

// Message has already been removed
```

```
// from the queue at this point.

// Process message
```

There is no need to call:

```
await receiver.CompleteMessageAsync(message);
```

because the message has already been deleted.

---

14. Complete Example – Peek

```
var message =
    await receiver.PeekMessageAsync();

Console.WriteLine(
    message.Body.ToString());
```

The message is only inspected.

Do **not** try:

```
await receiver.CompleteMessageAsync(message);
```

because the message was obtained using Peek, not through normal message processing.

---

15. Interview Questions

Q1. What are the two receive modes in Azure Service Bus?

Answer:

The two receive modes are:

1. **Peek-Lock**
2. **Receive-and-Delete**

Peek-Lock is the default mode.

Q2. What is the difference between Peek-Lock and Receive-and-Delete?

Answer:

In Peek-Lock mode, the message is locked when received but is not deleted until the receiver explicitly completes it.

In Receive-and-Delete mode, the message is deleted immediately when it is received.

Q3. Which mode is safer for important business messages?

Answer:

Peek-Lock is generally safer because the application can process the message and explicitly complete it only after successful processing.

Q4. What is the purpose of Peek?

Answer:

Peek is used to inspect a message without consuming or locking it for normal processing.

Q5. Can we complete a message received using Peek?

Answer:

No. Peek is only for inspection. A peeked message cannot be completed as a normally received, locked message.

---

16. Quick Revision

```
PEEK-LOCK
Receive
   ↓
Lock Message
   ↓
Process
```

```
   ↓
Complete
   ↓
Delete
```

```
RECEIVE-AND-DELETE
Receive
   ↓
Delete Immediately
   ↓
Process
```

```
PEEK
Peek
   ↓
Inspect
   ↓
Message remains
```

Easy Memory Trick

Peek-Lock = Look + Lock + Process + Complete

Receive-and-Delete = Receive + Delete

Peek = Just Look